

PHASE 1: Source Identification

***Goal:** Identify needed source documents that provide necessary knowledge and quality for a given RAG application.*

Core Concept: Corpus Composition

The foundational step of a Retrieval-Augmented Generation (RAG) pipeline is determining what data the Large Language Model (LLM) needs to answer domain-specific questions. In Databricks terminology, this is known as **Corpus composition and ingestion** [1].

The quality of the RAG application relies entirely on selecting the right data sources. For example, if you are building a customer support RAG bot, you must explicitly identify and aggregate FAQs, product manuals, and troubleshooting guides. An LLM cannot generate accurate, grounded responses if the provided context is missing important information or contains excessive, conflicting noise [2].

How it Works on Databricks

On the Databricks platform, identifying and preparing source documents involves integrating raw files into the unified governance framework:

- **Unity Catalog Volumes:** Non-tabular source documents (like PDFs, HTML, or Word docs) are typically uploaded to Unity Catalog Volumes, which provide secure, governed access to raw unstructured data.
- **Evaluation First:** Databricks emphasizes that identifying "quality" documents requires an evaluation framework. Before optimizing the pipeline, you should use Databricks Agent Evaluation (e.g., the `RetrievalGroundedness` judge) or MLflow to verify if the sources you identified actually provide adequate context to answer user queries [3].

When to Use vs. Alternatives

When identifying sources, you must classify them by structure to determine the downstream pipeline [4]:

- **Unstructured Data (PDFs, Wikis, Office Docs):** Use when the knowledge is stored in text-heavy formats. These documents require the full RAG data pipeline (parsing, chunking, and embedding into Databricks AI Search).

- **Structured Data (SQL databases, APIs):** Use when the required knowledge is tabular (e.g., customer records, transactional data). Instead of vectorizing this data, RAG agents can query structured data directly using tool calling or SQL generation.

Common Mistakes

- **Skipping Baseline Evaluation:** Databricks explicitly notes that *"Optimizing without measurement is guesswork."* [5] A major mistake is dumping thousands of documents into a vector index without setting up an automated way to measure if those documents improve or degrade answer quality.
- **Including Superfluous Information:** More data is not always better in RAG. Including irrelevant, outdated, or duplicated documents degrades the LLM's ability to find specific answers and increases the risk of hallucination.

Code Example: Ingesting Identified Sources

While identifying sources is a strategic decision, securely ingesting those identified documents into the Databricks ecosystem is the immediate next engineering step. Databricks uses **Auto Loader** to incrementally ingest unstructured source documents from a Unity Catalog Volume.

```
from pyspark.sql import SparkSession

# Ingesting identified source documents (e.g., PDFs) from a UC Volume
df = (spark.readStream
      .format("cloudFiles")
      .option("cloudFiles.format", "binaryFile")
      .load("/Volumes/main_catalog/rag_schema/source_documents")
)

# Write the raw documents to a Delta table for downstream processing
(df.writeStream
  .format("delta")
  .option("checkpointLocation", "/Volumes/main_catalog/rag_schema/
checkpoints/raw_docs")
  .table("main_catalog.rag_schema.raw_documents")
)
```

Function Explanation:

- `spark.readStream.format("cloudFiles")`: Invokes Databricks Auto Loader. This function automatically and efficiently detects new files in the specified Volume directory as you

identify and add new source documents, without requiring you to manually track which files were already processed.

- `option("cloudFiles.format", "binaryFile")`: Instructs Auto Loader to read the unstructured source documents as raw binary data. This preserves the exact file contents (vital for PDFs and images) so they can be properly parsed and chunked in Phase 2.

Sources (Official Databricks Documentation)

- [1] Build an unstructured data pipeline for RAG: <https://docs.databricks.com/aws/en/agents/tutorials/ai-cookbook/quality-data-pipeline-rag>
- [2] Improve RAG application quality: <https://docs.databricks.com/aws/en/agents/tutorials/ai-cookbook/quality-overview>
- [3] RetrievalGroundedness judge: https://docs.databricks.com/aws/en/mlflow3/genai/eval-monitor/concepts/judges/is_grounded
- [4] RAG on Databricks: <https://docs.databricks.com/aws/en/agents/retrieval-augmented-generation>
- [5] AI Search retrieval quality guide: <https://docs.databricks.com/aws/en/ai-search/retrieval-quality>